

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение
высшего образования

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»



Проект

По дисциплине: Имитационное моделирование
робототехнических систем

На тему: «Реализация и анализ имитационной модели
неполноприводного схвата робота KUKA youBot в среде
CoppeliaSim»

Студенты:
Рязанцев Д. Л.

Мирошниченко А. М.

Группа:

R4134с

Преподаватель:

Ракшин Е.А.

Санкт-Петербург, 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	5
Кинематика платформы всенаправленного движения youBot	5
Модель динамики робота youBot.....	7
СРЕДА ИССЛЕДОВАНИЯ	10
ХОД РАБОТЫ	13
1. Первое подключение.....	13
2. Реализация управления суставами робота youBot	17
3. Реализация функции управления схватом робота youBot.....	22
ЗАКЛЮЧЕНИЕ	29
СПИСОК ЛИТЕРАТУРЫ.....	30

ВВЕДЕНИЕ

Одной из ключевых задач при проектировании манипуляторов является создание захватных устройств (схватов), способных эффективно взаимодействовать с объектами в условиях неопределённости их формы, размеров и положения. Традиционные полноприводные схваты, где каждый степень свободы управляется отдельным приводом, обеспечивают высокую точность позиционирования, однако зачастую оказываются избыточно сложными, дорогими и энергозатратными для задач, где требуется не точный захват, а надёжная адаптация к форме объекта.

В этом контексте особый интерес представляют неполноприводные (адаптивные) схваты. Их принцип действия основан на использовании меньшего числа управляющих приводов, чем степеней свободы в механизме. За счёт введения пассивных (неуправляемых) сочленений, упругих элементов и особой кинематической структуры такие схваты способны автоматически подстраивать свою форму под контактируемый объект, обеспечивая стабильный и распределённый контакт.

В качестве эталонного объекта для изучения данных принципов широко используется роботизированная платформа KUKA youBot. Её двухпальцевый хват является классическим примером неполноприводного механизма: при четырёх степенях свободы (два сустава на каждый палец) он оснащён всего двумя приводами. Такая конфигурация представляет собой модель для исследования синергетического управления, анализа устойчивости захвата и сравнения с гипотетическими полноприводными аналогами.

Целью данного проекта является исследование и реализация имитационной модели робота KUKA youBot (в частности неполноприводного схвата) в среде CoppeliaSim. Работа направлена на экспериментальное исследование структуры механизма.

Основные задачи проекта включают: теоретическое исследование принципов работы адаптивных схватов; освоение инструментов удалённого

управления в CoppeliaSim через MATLAB-скрипты; реализацию алгоритмов управления для захвата объектов; проведение серии экспериментов; оформление полученных результатов в виде комплексного отчёта и презентации.

В основу научных исследований данной работы положены методы математического (обзор описания кинематики и динамики робота) и компьютерного моделирования (имитационное моделирование).

Результаты исследований могут послужить основой для выполнения лабораторных работ студентами младших курсов. Это позволит студентам быстрее изучить основы работы с KUKA youBot и перейти к решению более сложных задач на KUKA youBot.

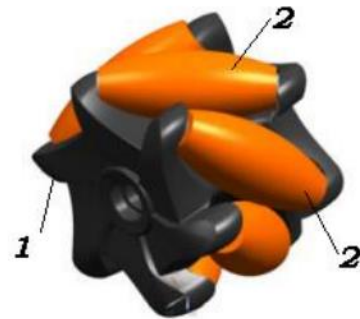
Объектом исследования является роботизированная мобильная платформа KUKA youBot, которая предназначена для решения исследовательских и промышленных задач.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

KUKA youBot – мобильная платформа всенаправленного движения с механумколёсами (Рис. 1.), оснащённая одним или двумя манипуляторами. Для этого робота создано свободное и открытое программное обеспечение, что позволяет использовать youBot для широкого класса научно-исследовательских и учебных задач.



а) Общий вид робота с одним манипулятором с устройством захвата



б) Роликонесущее механум-колесо
(1 – диск колеса, 2 - ролики)

Рисунок 1. Мобильный робот KUKA youBot

Кинематика платформы всенаправленного движения youBot

Исследуем движение мобильного робота youBot по горизонтальной плоскости (подстилающей поверхности, полу). Считаем, что манипулятор неподвижен относительно платформы. Для описания кинематики платформы мобильного робота введём неподвижную систему координат $X'Y'Z'$ с горизонтальной плоскостью $X'Y'$, и связанную с платформой подвижную $OXYZ$ с началом в геометрическом центре платформы (Рис. 2).

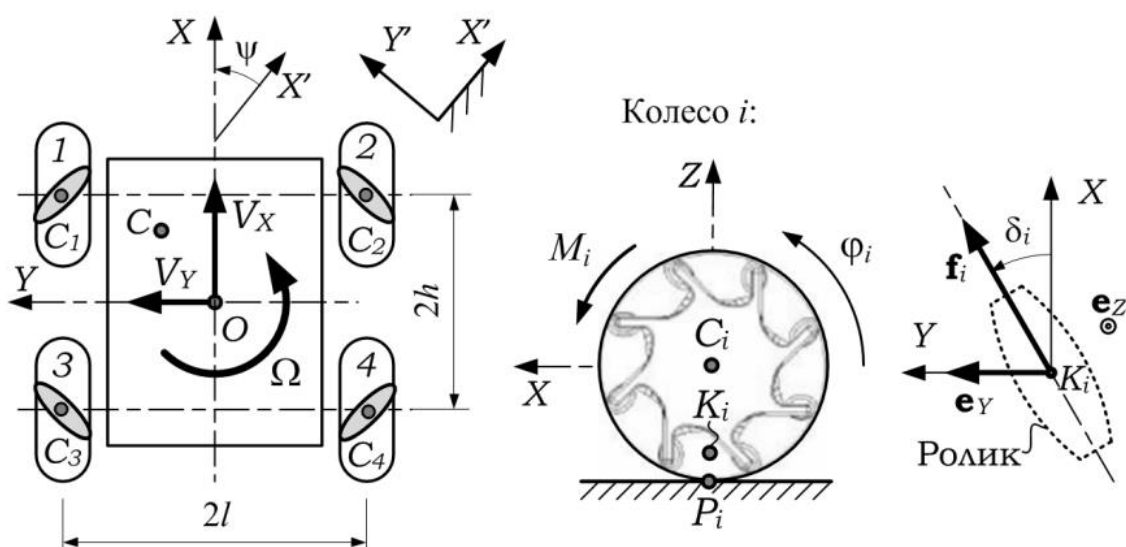


Рисунок 2. Схема мобильного робота KUKA youBot

Движение системы описывается обобщёнными координатами

Декартовыми координатами XO' и YO' её геометрического центра O ; курсового угла $\psi = \angle(X', X)$; углами поворота колёса относительно корпуса робота ϕ_i ($i = 1, \dots, 4$); углами поворота роликов, контактирующих с подстилающей поверхностью, относительно дисков колёс γ_i ($i = 1, \dots, 4$). В рамках настоящей работы рассматривается идеальная модель движения механум-колес (омни-колес). Считаем, что в каждый момент времени ролик колеса i движется без проскальзывания и отрыва от подстилающей поверхности, причём центр колеса C_i , центр оси ролика K_i и точка контакта P_i ролика с полом лежат на одной вертикальной прямой (Рис. 2). В соответствии с принятыми допущениями вектор скорости точки контакта:

$$\vec{V}_{P_i} = \vec{V}_O + [\vec{\Omega}, \overrightarrow{OC_i}] + [\vec{\omega}_i, \overrightarrow{C_iK_i}] + [\vec{\omega}_{p_i}, \overrightarrow{K_iP_i}] = 0,$$

где $\vec{V}_O = \{V_X, V_Y, 0\}$ - вектор скорости центра платформы; $\vec{\Omega}$, $\vec{\omega}_i$ и $\vec{\omega}_{p_i}$ – соответственно, векторы абсолютных угловых скоростей платформы, колеса i и его ролика, контактирующего с полом; $[,]$ - операция векторного умножения. Перечисленные векторы угловых скоростей равны:

$$\vec{\Omega} = \Omega \vec{e}_Z, \vec{\omega}_i = \Omega \vec{e}_Z + \dot{\phi}_i \vec{e}_Y, \vec{\omega}_{pi} = \Omega \vec{e}_Z + \dot{\phi}_i \vec{e}_Y + \dot{\gamma}_i \vec{f}_i,$$

Выполнив необходимые преобразования для каждого из колёс ($i = 1, \dots, 4$), из данных формул и получаем выражения для угловых скоростей вращения колёс и роликов:

$$\begin{aligned} \dot{\phi}_1 &= \frac{1}{R} [V_X - V_Y - (l + h)\Omega], \quad \dot{\phi}_2 = \frac{1}{R} [V_X + V_Y + (l + h)\Omega], \quad \dot{\gamma}_1 = \dot{\gamma}_2 = -\frac{\sqrt{2}}{r} [V_Y + h\Omega] \\ \dot{\phi}_3 &= \frac{1}{R} [V_X + V_Y - (l + h)\Omega], \quad \dot{\phi}_4 = \frac{1}{R} [V_X - V_Y + (l + h)\Omega], \quad \dot{\gamma}_3 = \dot{\gamma}_4 = -\frac{\sqrt{2}}{r} [V_Y - h\Omega] \end{aligned}$$

где $R = |\mathbf{CiPi}|$ - радиус механум-колеса, $r = |\mathbf{KiPi}|$ - радиус поперечного сечения ролика, l и h - характерные размеры платформы (Рис. 2). Полученные соотношения обосновывают всенаправленное свойство движения платформы – произвольные законы изменения V_X , V_Y и Ω могут быть реализованы за счёт вращения роликонесущих колёс. Например, для поступательного движения робота вбок ($V_X = 0$, $\Omega = 0$) вращение колёс должно производиться со скоростями $\dot{\phi}_2 = \dot{\phi}_3 = -\dot{\phi}_1 = -\dot{\phi}_4 = V_Y/R$, а для вращения вокруг центра платформы ($V_X = 0$, $V_Y = 0$) – со скоростями

$$\dot{\phi}_2 = \dot{\phi}_4 = -\dot{\phi}_1 = -\dot{\phi}_3 = (l + h)\Omega/R.$$

Модель динамики робота youBot

Составим уравнения движения робота, считая, что в сочленениях тел действуют линейные по скорости силы вязкого трения, а ролики и вращающиеся детали механических передач колёс невесомы. Каждое колесо рассматриваемой системы оснащено отдельным электроприводом. Моменты, развиваемые этими приводами обозначим $M_i (i = 1, \dots, 4)$ (Рис. 2). Соотношения определяют восемь неголономных связей, налагаемых на динамику мобильного робота youBot. Из его 11 обобщённых скоростей можно составить лишь три независимые линейные комбинации, например,

$$V_X \equiv \dot{X}'_O \cos\psi + \dot{Y}'_O \sin\psi, V_Y \equiv -\dot{X}'_O \sin\psi + \dot{Y}'_O \cos\psi, \Omega \equiv \dot{\psi},$$

и с помощью них описать динамику системы. Для составления уравнений движения робота воспользуемся уравнениями Аппеля. Построение уравнений Аппеля для исследуемой системы подробно изложено в работе. С учётом введённых выше допущений, динамика мобильного робота youBot описывается следующей системой уравнений:

$$\begin{aligned} m_0(\dot{V}_X - V_Y\Omega) + \frac{4I_1\dot{V}_X}{R^2} - ma_Y\dot{\Omega} - ma_X\Omega^2 + \frac{4\mu_1V_X}{R^2} &= F_X, \\ m_0(\dot{V}_Y + V_X\Omega) + \frac{4I_1\dot{V}_Y}{R^2} + ma_X\dot{\Omega} - ma_Y\Omega^2 + \left(\frac{4\mu_1}{R^2} + \frac{8\mu_2}{r^2}\right)V_Y &= F_Y, \\ \left(I_0 + \frac{4I_1(l+h)^2}{R^2}\right)\dot{\Omega} - ma_Y(\dot{V}_X - V_Y\Omega) + ma_X(\dot{V}_Y + V_X\Omega) + \left(\frac{4\mu_1(l+h)^2}{R^2} + \frac{8\mu_2h^2}{r^2}\right)\Omega &= M_\Omega, \end{aligned}$$

где $m_0 = m + 4m_1$ – общая масса робота; I_0 – его момент инерции относительно оси OZ; m – масса платформы со всем перевозимым оборудованием и грузом; a_X и a_Y – координаты её центра масс C в подвижной системе OXY (Рис. 2); I_1 – момент инерции колеса относительно его оси; m_1 – масса колеса; μ_1 – коэффициент вязкого трения в сочленениях колёс и платформы; μ_2 – коэффициент вязкого трения в осях роликов; F_X , F_Y и M_Ω – управляющие обобщённые силы, которые выражаются через моменты приводов колёс M_i , $i = 1...4$, по формулам

$$\begin{aligned} F_X &= \frac{M_2 + M_1 + M_4 + M_3}{R}, F_Y = \frac{M_2 - M_1 - M_4 + M_3}{R}, M_\Omega \\ &= \eta_1[M_2 - M_1 + M_4 - M_3]. \end{aligned}$$

Значения некоторых коэффициентов в системе неизвестны и подлежат оцениванию. Для удобства дальнейшего анализа, введём новые переменные:

$$V_X = (l + h)\Omega_{V_X}, V_Y = (l + h)\Omega_{V_Y}, M_{V_X} = (l + h)F_X, M_{V_Y} = (l + h)F_Y,$$

и перепишем систему уравнений движения в следующей форме:

$$y = H x,$$

где

$$\begin{pmatrix} m_0(l+h)^2 & I_0 & \frac{4I_1(l+h)^2}{R^2} & m_{aX}(l+h) & m_{aY}(l+h) & \frac{4\mu_1(l+h)^2}{R^2} & \frac{8\mu_2 h^2}{r^2} \end{pmatrix}^T -$$

вектор неизвестных постоянных параметров (коэффициентов в уравнениях движения);

$$y = \begin{pmatrix} M_{V_X} \\ M_{V_Y} \\ M_{\Omega} \end{pmatrix}, H$$

$$= \begin{bmatrix} \dot{\Omega}_{V_X} - \Omega_{V_Y} \Omega & 0 & \dot{\Omega}_{V_X} & -\Omega^2 & -\dot{\Omega} & \Omega_{V_X} & 0 \\ \dot{\Omega}_{V_Y} + \Omega_{V_X} \Omega & 0 & \dot{\Omega}_{V_Y} & \dot{\Omega} & -\Omega^2 & \Omega_{V_Y} & \eta^2 \Omega_{V_Y} \\ 0 & \dot{\Omega} & \dot{\Omega} & \Omega_{V_X} \Omega + \dot{\Omega}_{V_Y} & \Omega_{V_Y} \Omega - \dot{\Omega}_{V_X} & \Omega & \Omega \end{bmatrix};$$

$\eta = (l+h)/h$ – безразмерная длина.

СРЕДА ИССЛЕДОВАНИЯ

Современные инструменты имитационного моделирования, такие как среда CoppeliaSim, предоставляют уникальную возможность глубокого и безопасного изучения подобных систем. Они позволяют не только воспроизвести физику взаимодействия, но и провести масштабные виртуальные эксперименты, количественно оценивая ключевые характеристики механизма до его физической реализации.

Список узлов из робототехнической симуляции в среде CoppeliaSim, описывающий структуру робота youBot представлен на рисунке 3.

Модель начинается с корневого объекта youBot, который служит абсолютным родителем для всех остальных компонентов робота в симуляционном пространстве. Он не несёт физической геометрии, но задаёт общую систему координат и является точкой сборки всей конструкции. Непосредственно под ним располагается система мобильного шасси, которое теперь представлено полноценной моделью с колёсами и приводами.

Система передвижения состоит из четырёх идентичных, но независимых модулей, по одному на каждое колесо: переднее левое (fl), переднее правое (fr), заднее правое (rr) и заднее левое (rl). Каждый модуль – это законченный кинематический узел. Его сердцем является активный привод `rollingJoint_fl` (и аналогично `_fr`, `_rr`, `_rl`). Это вращательное соединение, которое непосредственно раскручивает колесо вокруг его оси, обеспечивая основную тягу. Управляющая логика для каждого привода вложена в дочерний объект `Script`. Далее идёт `intermediateLink_fl` – модель колеса fl соответственно.

Ключевым элементом модуля является `swedishWheel_fl` – это визуальная и кинематическая модель шведского омни-колеса. Его уникальная конструкция с роликами, расположенными под углом 45 градусов к ободу, позволяет ему не только катиться вперёд-назад от основного привода, но и свободно скользить вбок. Именно для моделирования этого бокового скольжения используется специальное

соединение `slippingJoint_fl`. Оно не является приводным, а представляет собой пассивную степень свободы, позволяющую колесу перемещаться под углом относительно своей оси вращения. Эта комбинация активного `rollingJoint` и пассивного `slippingJoint` и есть основа кинематики омни-колеса. Завершает модуль объект `wheel_respondable_fl` – это физическое тело колеса, наделённое свойствами массы, инерции и коллизии. Именно этот объект сталкивается с полом и другими объектами в симуляции, превращая математическую модель в физически взаимодействующий элемент.

Все четыре таких модуля, смонтированные по углам прямоугольного корпуса `Rectangle0`, и образуют платформу `youBot`. Общий объект `Joint` на верхнем уровне является виртуальным соединением, объединяющим или синхронизирующим эти четыре привода для упрощения управления.

На этой мобильной базе установлена верхняя платформа, представленная объектом `ME_Plato2_sub1` и дополненная геометрическими элементами `Rectangle` и `Rectangle13`. Здесь же находится `visionSensor` – основной оптический датчик робота.

На платформе смонтирован пятизвенный манипулятор. Его кинематическая цепь построена на суставах от `youBotArm.Joint0` до `youBotArm.Joint4`. `Joint0` – это поворотное основание. `Joint1` и `Joint2` – это основные плечевые суставы. `Joint3` выполняет роль локтя, а `Joint4` – схват. Каждому суставу соответствует механическое звено (например, `ME_Arm1_m0_sub0_sub0` для первого) и геометрическая форма (цилиндры `Cylinder`, `Cylinder1`, `Cylinder2` или прямоугольники), которые составляют видимую часть руки. Датчик `youBot_orientationBase` отслеживает пространственную ориентацию системы.

Рабочий орган – двухпальцевый, который представлен кистью `ME_Arm5_me_sub1` и двумя призматическими суставами: `youBotGripper.Joint1` и `youBotGripper.Joint2`. Каждый палец составлен из двух прямоугольных сегментов (`Rectangle5` + `Rectangle2` и `Rectangle6` + `Rectangle3`), что позволяет реалистично имитировать захват.

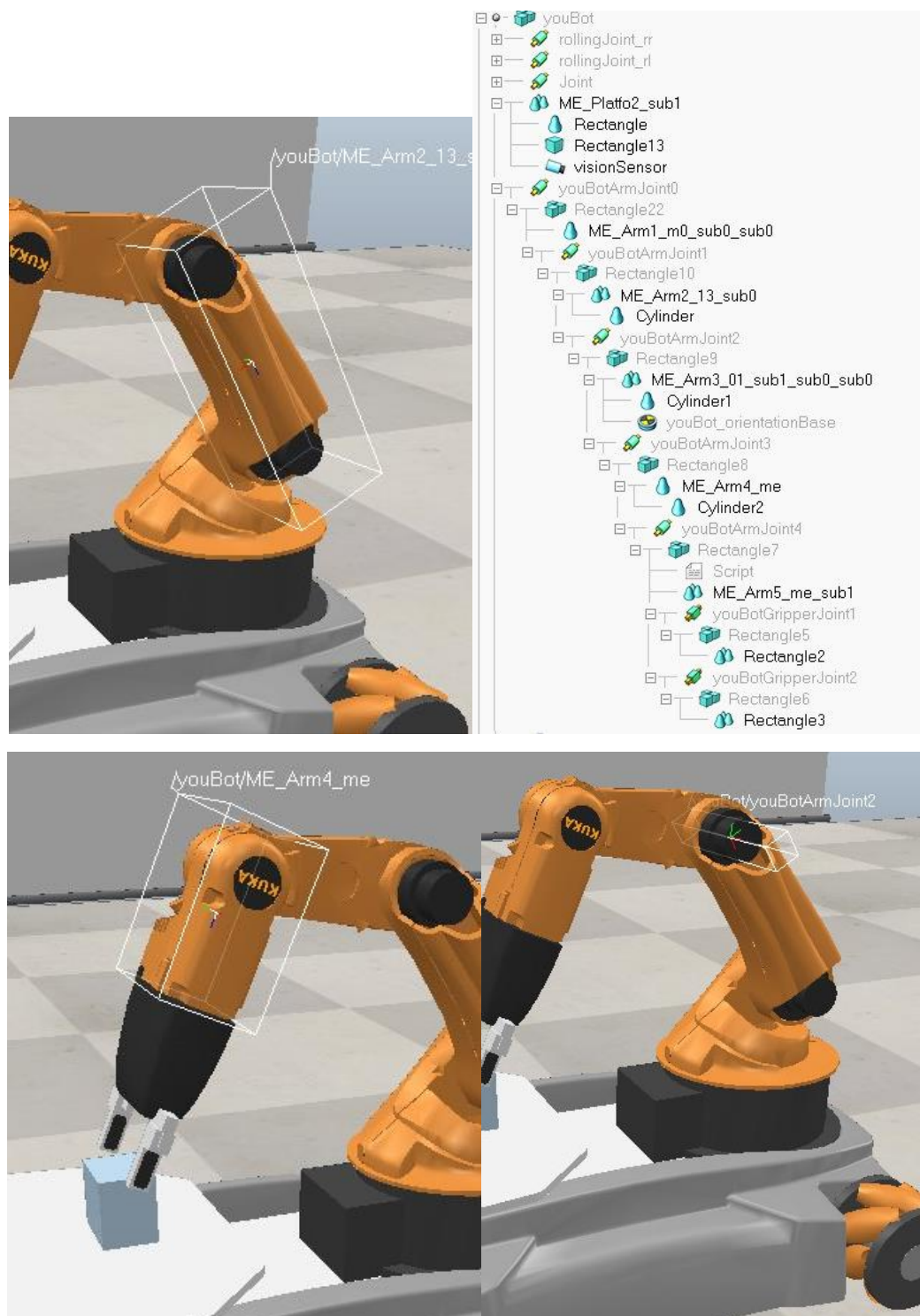


Рисунок 3. Некоторые узлы робототехнической симуляции в среде CoppeliaSim и структура робота youBot

ХОД РАБОТЫ

1. Первое подключение

Главный скрипт, который обеспечивает подключение среды MATLAB к симулятору CoppeliaSim, инициализацию модели робота KUKA YouBot и запуск демонстрации представлен на рисунке 4. Скрипт начинает работу с определения геометрических параметров манипулятора – длин его звеньев. Затем с использованием Remote API CoppeliaSim устанавливается сетевое соединение с симулятором, запущенным на локальном компьютере. После успешного подключения скрипт в цикле получает дескрипторы виртуальных объектов, соответствующих каждому управляемому сочленению робота, по их уникальным именам в сцене. Эти дескрипторы сохраняются в массив и вместе с остальными параметрами передаются в функцию `demoVelocityKinematics`, которая будет выполнять логику демонстрации. Данный скрипт является связующим звеном между алгоритмической частью проекта, реализованной в MATLAB, и виртуальным прототипом робота в CoppeliaSim, позволяя наглядно тестировать и демонстрировать работу алгоритмов кинематики.

Реализация данного скрипта является важной частью проекта, поскольку он обеспечивает фундаментальную связь между теоретическими алгоритмами (MATLAB) и их практической проверкой в виртуальной среде (CoppeliaSim). Без корректного подключения, получения дескрипторов сочленений и передачи управления невозможно было бы продемонстрировать работу алгоритмов кинематики на модели реального робота. Этап первого подключения служит техническим мостом, который трансформирует математические расчёты в визуализируемое и контролируемое движение манипулятора, что является обязательным условием для всей последующей демонстрации разработанных методов.

```

L = [0.147, 0.155, 0.135, 0.081, 0.137];

sim = remApi('remoteApi');
sim.simxFinish(-1);
clientID = sim.simxStart('127.0.0.1', 19999, true, true, 5000, 5);

if clientID > -1
    disp('Connected to CoppeliaSim');

    joint_handles = zeros(1,5);
    joint_names = {'youBotArmJoint0', 'youBotArmJoint1', 'youBotArmJoint2', 'youBotArmJoint3', 'youBotArmJoint4'};
    for i = 1:5
        [~, joint_handles(i)] = sim.simxGetObjectHandle(clientID, joint_names{i}, sim.simx_opmode_blocking);
    end

    demoVelocityKinematics(sim, clientID, joint_handles, L);

    sim.simxFinish(clientID);
else
    disp('Connection failed');
end

```

Рисунок 4. Параметры KUKA youBot

Функция `demoVelocityKinematics`, организующая демонстрацию работы алгоритмов скоростного управления роботом-манипулятором в среде симуляции представлена на рисунке 5. В начале функция получает начальные углы сочленений через API симулятора и вычисляет исходное положение схвата средствами прямой кинематики. Затем выполняется первая часть демонстрации – прямая задача: задаётся вектор скорости схвата, направленный строго вниз, и с помощью функции `applySmoothVelocity` происходит плавное преобразование этой декартовой скорости в скорости сочленений через матрицу Якоби и их отработка в симуляторе. После паузы запускается вторая часть – обратная задача: функция `returnToInitialCartesian` осуществляет возврат схвата в исходную точку, используя алгоритм, основанный на обращении матрицы Якоби для вычисления требуемых скоростей в сочленениях на каждом шагу движения. В завершение функция вычисляет и отображает итоговую позицию и ошибку позиционирования, демонстрируя точность работы всей системы.

```

function demoVelocityKinematics(sim, clientID, joint_handles, L)
    fprintf('\n=== КИНЕМАТИКА ПО СКОРОСТИ ===\n');

    theta_initial = getJointAngles(sim, clientID, joint_handles);
    [~, initial_pos, ~] = forwardKinematics(theta_initial, L);
    fprintf('Начальные углы: [%2f, %2f, %2f, %2f, %2f] рад\n', theta_initial);
    fprintf('Начальная позиция КЗ: [%3f, %3f, %3f] м\n', initial_pos);

    % 1. ПРЯМАЯ ЗАДАЧА: наклон вперед
    fprintf('\n1. ПРЯМАЯ ЗАДАЧА: наклон вперед\n');

    v_forward = [0.0; 0.0; -0.02; 0.0; 0.0; 0.0];

    applySmoothVelocity(sim, clientID, joint_handles, theta_initial, L, v_forward, 3);

    theta_current = getJointAngles(sim, clientID, joint_handles);
    [~, current_pos, ~] = forwardKinematics(theta_current, L);
    fprintf('Позиция после наклона: [%3f, %3f, %3f] м\n', current_pos);

    pause(1);

    % 2. ОБРАТНАЯ ЗАДАЧА: возврат в исходное положение
    fprintf('\n2. ОБРАТНАЯ ЗАДАЧА: возврат в исходное положение\n');

    returnToInitialCartesian(sim, clientID, joint_handles, theta_current, initial_pos, L, 4);

    theta_final = getJointAngles(sim, clientID, joint_handles);
    [~, final_pos, ~] = forwardKinematics(theta_final, L);
    fprintf('\nФинальная позиция: [%3f, %3f, %3f] м\n', final_pos);
    fprintf('Ошибка позиции: %.4f м\n', norm(initial_pos - final_pos));

    fprintf('\nДемонстрация завершена\n');
end

```

Рисунок 5. Функция кинематики по скорости, в которой описано решение прямой и обратной задачи кинематики по скорости

Реализация функции `computeJacobian`, написанной на MATLAB, которая вычисляет геометрическую матрицу Якоби для пятизвенного манипулятора представлена на рисунке 6. Функция принимает на вход текущие значения углов в сочленениях и длины звеньев, а затем формирует матрицу 6×5 , связывающую скорости изменения обобщённых координат с линейной и угловой скоростью конечного звена. Внутри функции последовательно вычисляются однородные матрицы преобразования для каждого звена в соответствии с параметрами Денавита – Хартенберга, определяются положения всех сочленений и направления их осей вращения, после чего матрица Якоби заполняется по классическим формулам: для линейной скорости используется векторное произведение оси вращения сочленения на вектор от этого сочленения к схвату, а для угловой скорости – непосредственно сама ось вращения. Данная функция является

математической основой, обеспечивающей решение как прямой, так и обратной задач скоростной кинематики в рамках проекта.

```
function J = computeJacobian(theta, L)
% Вычисление геометрического якобиана
J = zeros(6, 5);
alpha = [pi/2, 0, 0, pi/2, 0];
a = [0, L(2), L(3), 0, 0];
d = [L(1), 0, 0, 0, L(5)];

T = eye(4);
z_axes = zeros(3, 6);
positions = zeros(3, 6);

z_axes(:, 1) = [0; 0; 1];
positions(:, 1) = [0; 0; 0];

for i = 1:5
    ct = cos(theta(i)); st = sin(theta(i));
    ca = cos(alpha(i)); sa = sin(alpha(i));

    T_i = [ct, -st*ca, st*sa, a(i)*ct;
           st, ct*ca, -ct*sa, a(i)*st;
           0, sa, ca, d(i);
           0, 0, 0, 1];

    T = T * T_i;
    z_axes(:, i+1) = T(1:3, 3);
    positions(:, i+1) = T(1:3, 4);
end

p_end = positions(:, 6);

for i = 1:5
    J(1:3, i) = cross(z_axes(:, i), p_end - positions(:, i));
    J(4:6, i) = z_axes(:, i);
end
end
```

Рисунок 6. Функция, описывающая вычисление Якобиана

На рисунке 7 представлен консольный вывод, полученный в результате выполнения программы. В нём отражён полный цикл работы: начальное состояние робота (углы сочленений и положение схвата), поэтапное выполнение прямого движения (наклона вперёд) с отображением текущей позиции схвата на ключевых шагах, процесс возврата в исходную точку с указанием уменьшающейся ошибки позиционирования на каждом контрольном шаге, а также финальные результаты – достигнутое положение схвата и остаточная ошибка. Вывод наглядно показывает, что алгоритм справился с задачами: при наклоне вперёд координата Z схвата монотонно уменьшалась при постоянных X и Y , а в ходе обратного движения ошибка позиции плавно сошла с 3 см до 4 мм. Ошибка финальной позиции в 4.1 мм свидетельствует о практической работоспособности реализованных методов скоростной кинематики с учётом дискретности управления и численных

погрешностей.

```
=== КИНЕМАТИКА ПО СКОРОСТИ ===
Начальные углы: [0.00, 0.54, 0.92, 1.27, -0.00] рад
Начальная позиция КЭ: [0.204, 0.000, 0.486] м

1. ПРЯМАЯ ЗАДАЧА: наклон вперед
Плавное движение...
Шаг 20/60: позиция [0.204, 0.000, 0.475] м
Шаг 40/60: позиция [0.204, 0.000, 0.454] м
Шаг 60/60: позиция [0.204, 0.000, 0.444] м
Движение завершено
Позиция после наклона: [0.204, 0.000, 0.444] м

2. ОБРАТНАЯ ЗАДАЧА: возврат в исходное положение
Возврат в исходную позицию...
Шаг 20/80: ошибка позиции 0.0308 м
Шаг 40/80: ошибка позиции 0.0137 м
Шаг 60/80: ошибка позиции 0.0061 м
Шаг 80/80: ошибка позиции 0.0043 м

Финальная позиция: [0.204, 0.000, 0.482] м
Ошибка позиции: 0.0041 м

Демонстрация завершена
```

Рисунок 7. Результат выполнения программы

В ходе работы, на этапе первого подключения, был реализован и выполнен начальный этап проекта – установление связи между MATLAB и средой CoppeliaSim. Была создана и запущена программа, которая установила устойчивое соединение между вычислительной средой и виртуальным роботом через Remote API, корректно идентифицировала и получила управление над всеми сочленениями манипулятора KUKA YouBot, а также передала управление демонстрационной функции

2. Реализация управления суставами робота youBot

Код, представленный на рисунке 8 нужен для того, чтобы подключиться к симулятору CoppeliaSim и получить полный контроль над роботом YouBot. С помощью него создается связь между MATLAB и симулятором, находятся все основные компоненты робота - его четыре колеса для движения, пять суставов манипулятора для работы рукой, два пальца захвата (gripper) для удержания предметов и камеру для зрения - и

происходит сохранение их идентификаторов. Эти идентификаторы (handle), работают как ярлыки, которые позволяют управлять каждым элементом робота, не запоминая их названия в симуляторе. После успешного подключения и получения всех идентификаторов код запускает графический интерфейс, через который реализовано управление роботом в реальном времени: задавать скорость колес, позиционировать руку с помощью суставов.

```
handles = [0,0,0,0,0,0,0,0,0,0,0,0];

%joint handles
[~, handles(1)]=sim.simxGetObjectHandle(clientID,'rollingJoint_rr',sim.simx_opmode_blocking);
[~, handles(2)]=sim.simxGetObjectHandle(clientID,'rollingJoint_rl',sim.simx_opmode_blocking);
[~, handles(3)]=sim.simxGetObjectHandle(clientID,'rollingJoint_fr',sim.simx_opmode_blocking);
[~, handles(4)]=sim.simxGetObjectHandle(clientID,'rollingJoint_fl',sim.simx_opmode_blocking);
[~, handles(5)]=sim.simxGetObjectHandle(clientID,'youBotArmJoint0',sim.simx_opmode_blocking);
[~, handles(6)]=sim.simxGetObjectHandle(clientID,'youBotArmJoint1',sim.simx_opmode_blocking);
[~, handles(7)]=sim.simxGetObjectHandle(clientID,'youBotArmJoint2',sim.simx_opmode_blocking);
[~, handles(8)]=sim.simxGetObjectHandle(clientID,'youBotArmJoint3',sim.simx_opmode_blocking);
[~, handles(9)]=sim.simxGetObjectHandle(clientID,'youBotArmJoint4',sim.simx_opmode_blocking);
[~, handles(10)]=sim.simxGetObjectHandle(clientID,'youBotGripperJoint1',sim.simx_opmode_blocking);
[~, handles(10)]=sim.simxGetObjectHandle(clientID,'youBotGripperJoint2',sim.simx_opmode_blocking);
[~, handles(11)]=sim.simxGetObjectHandle(clientID,'visionSensor',sim.simx_opmode_blocking);
```

Рисунок 8. Получение идентификаторов компонентов робота в файле main.m

Для осуществления управления роботом был создан файл gui.m, где были реализованы функции управления звеньями с помощью идентификаторов, также реализовано окно MATLAB App, представленное на рисунке 9, через которое и происходит управление роботом в реальном времени.

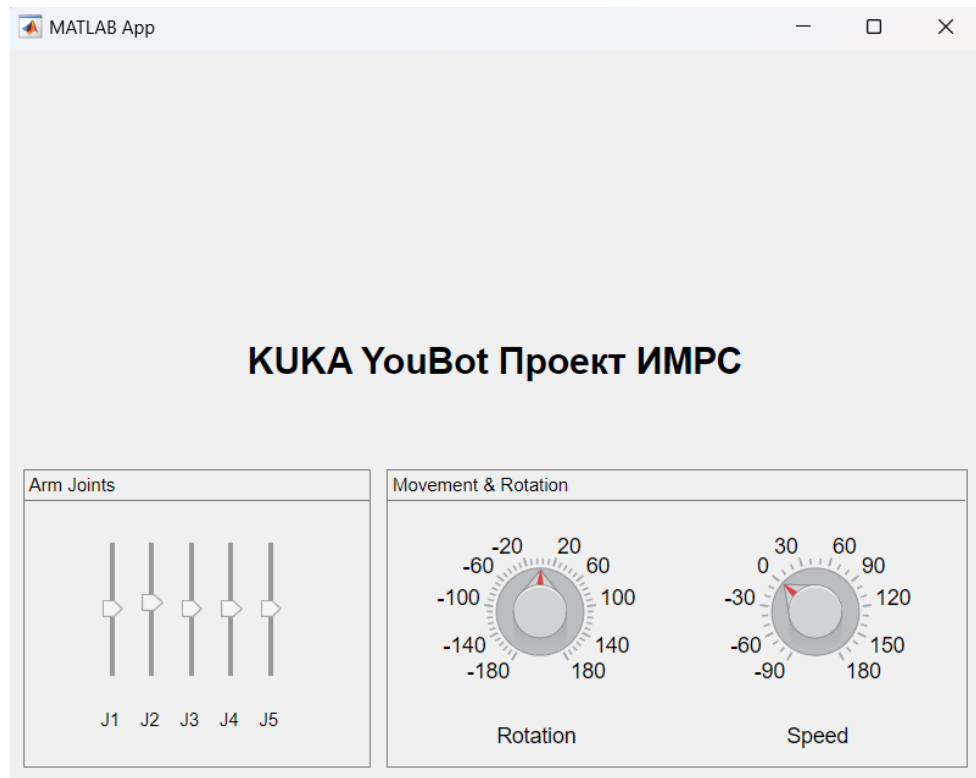


Рисунок 9. Окно MATLAB App

Код, отвечающий за управление движением робота через два основных режима: линейное перемещение вперед и назад, а также вращение на месте, представлен на рисунке 10. Когда пользователь регулирует ползунок скорости, код передает всем четырем колесам одинаковое значение скорости, заставляя робота двигаться прямо или назад в зависимости от направления. При управлении вращением код задает противоположные скорости для правых и левых колес, что заставляет робота поворачиваться вокруг своей оси.

```
function RotationKnobValueChanged(app, event)
    newValue = event.Value;
    absNewValue = abs(newValue);

    % Prevents low value close to 0 not fully stoppin the wheels.
    if newValue < 10 && newValue > -10
        resetWheelVelocities(app)
        return
    end

    if newValue < 0
        setWheelJointVelocity(app,1,absNewValue);
        setWheelJointVelocity(app,2,-absNewValue);
        setWheelJointVelocity(app,3,absNewValue);
        setWheelJointVelocity(app,4,-absNewValue);
    elseif newValue > 0
        setWheelJointVelocity(app,1,-absNewValue);
        setWheelJointVelocity(app,2,absNewValue);
        setWheelJointVelocity(app,3,-absNewValue);
        setWheelJointVelocity(app,4,absNewValue);
    end
end

function SpeedKnobValueChanged(app, event)
    newValue = event.Value;
    setWheelJointVelocity(app,1,newValue);
    setWheelJointVelocity(app,2,newValue);
    setWheelJointVelocity(app,3,newValue);
    setWheelJointVelocity(app,4,newValue);
end
```

Рисунок 10. Реализация функций движения и вращения колес в файле gui.m

Код на рисунке 11 позволяет управлять рукой робота, преобразуя движения пяти ползунков в графическом интерфейсе в реальные углы поворота суставов манипулятора. Каждый ползунок отвечает за конкретный сустав руки: от основания до захвата. При перемещении ползунка, код отправляет команду в симулятор CoppeliaSim для установки нужного угла поворота соответствующего сустава, обеспечивая точное позиционирование манипулятора для выполнения различных задач.

```
% Value changing function: jointOneSlider
function jointOneSliderValueChanging(app, event)
    changingValue = event.Value;
    setJointTargetPosition(app, 5, changingValue);
end

% Value changing function: jointTwoSlider
function jointTwoSliderValueChanging(app, event)
    changingValue = event.Value;
    setJointTargetPosition(app, 6, changingValue);
end

% Value changing function: jointThreeSlider
function jointThreeSliderValueChanging(app, event)
    changingValue = event.Value;
    setJointTargetPosition(app, 7, changingValue);
end

% Value changing function: jointFourSlider
function jointFourSliderValueChanging(app, event)
    changingValue = event.Value;
    setJointTargetPosition(app, 8, changingValue);
end

% Value changing function: jointFiveSlider
function jointFiveSliderValueChanging(app, event)
    changingValue = event.Value;
    setJointTargetPosition(app, 9, changingValue);
end

% Callback для захвата
function GripperSliderValueChanging(app, event)
    changingValue = event.Value;
    setGripperPosition(app, changingValue);
end

function setJointTargetPosition(app, index, newValue)
    app.sim.simxSetJointTargetPosition( ...
        app.clientID, ...
        app.handles(index), ...
        getRadiansForDegrees(app,newValue), ...
        app.sim.simx_opmode_streaming ...
    );
end
```

Рисунок 11. Реализация функций управления суставами руки в файле gui.m

На рисунках 12 – 13 приведена демонстрация работы программы. В первом случае приведено начальное положение робота: объект стоит на расстоянии от стула, расположен прямо, а также конечное положение: робот переместился назад к стулу и расположен под углом 45°.

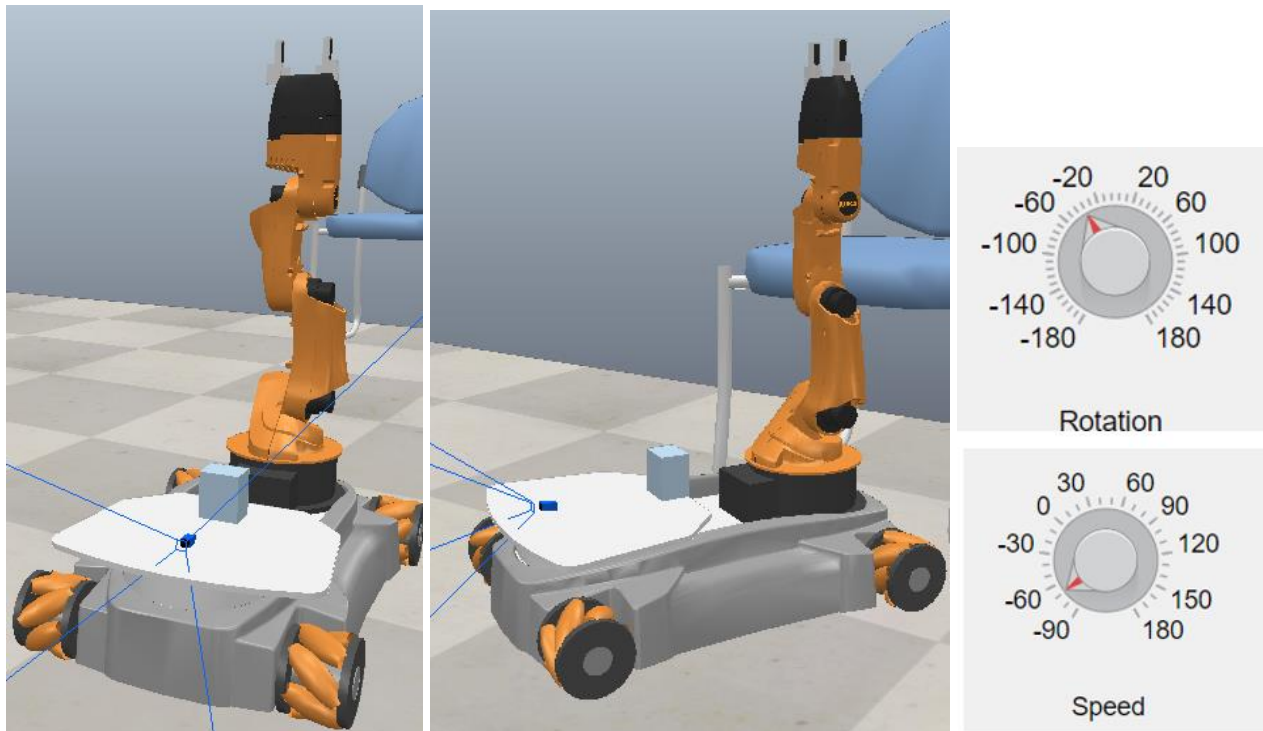


Рисунок 12. Демонстрация управления платформой робота

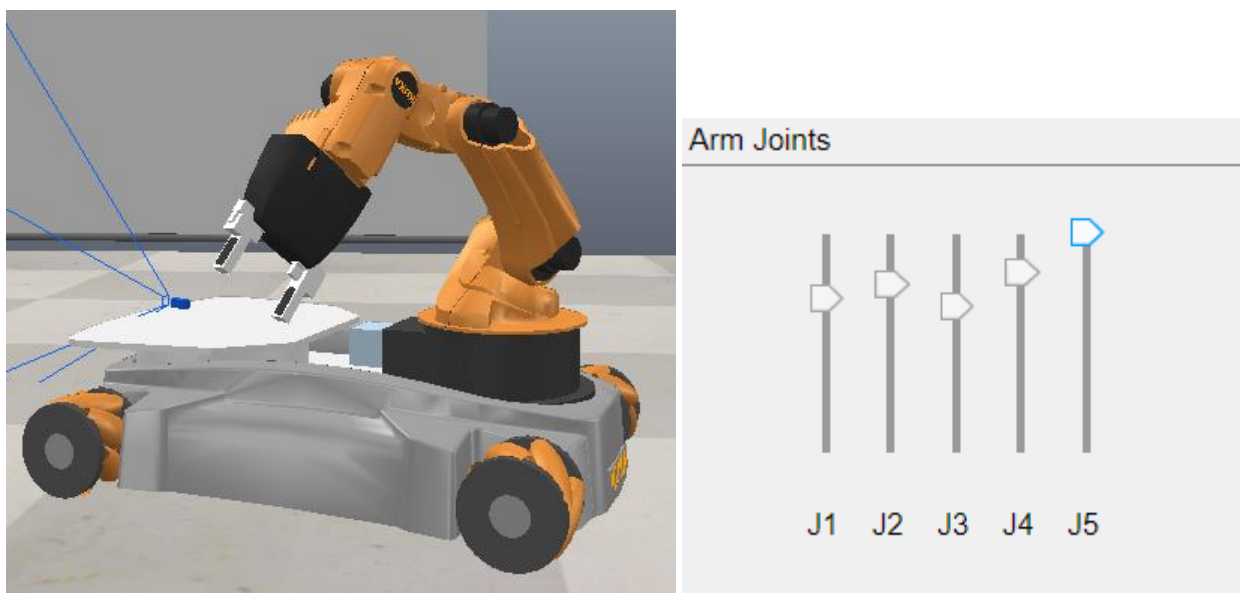


Рисунок 13. Демонстрация управления суставами руки робота

В заключении, можно отметить, что на данном этапе работы была разработана и реализована система управления роботом youBot в симуляционной среде CoppeliaSim через MATLAB. Основное достижение – создание контура управления, включающего подключение к симулятору, идентификацию всех компонентов робота и разработку графического интерфейса. Это позволило получить практический инструмент для управления движением платформы и позиционированием манипулятора, что необходимо для дальнейшего изучения и отработки алгоритмов управления

схватом данного робота, а также планирования траекторий и выполнения конкретных задач без риска повреждения физического оборудования.

3. Реализация функции управления схватом робота youBot

Для реализации функции управления схватом робота необходимо скорректировать часть кода в файле `main.m`, отвечающую за идентификаторы. Это нужно для того, чтобы разделить пальцы манипулятора друг от друга и получить возможность управления ими по-отдельности. Исправленный код представлен на рисунке 14.

```
handles = [0,0,0,0,0,0,0,0,0,0,0,0];

% joint handles
[~, handles(1)]=sim.simxGetObjectHandle(clientID,'rollingJoint_rr',sim.simx_opmode_blocking);
[~, handles(2)]=sim.simxGetObjectHandle(clientID,'rollingJoint_rl',sim.simx_opmode_blocking);
[~, handles(3)]=sim.simxGetObjectHandle(clientID,'rollingJoint_fr',sim.simx_opmode_blocking);
[~, handles(4)]=sim.simxGetObjectHandle(clientID,'rollingJoint_fl',sim.simx_opmode_blocking);
[~, handles(5)]=sim.simxGetObjectHandle(clientID,'youBotArmJoint0',sim.simx_opmode_blocking);
[~, handles(6)]=sim.simxGetObjectHandle(clientID,'youBotArmJoint1',sim.simx_opmode_blocking);
[~, handles(7)]=sim.simxGetObjectHandle(clientID,'youBotArmJoint2',sim.simx_opmode_blocking);
[~, handles(8)]=sim.simxGetObjectHandle(clientID,'youBotArmJoint3',sim.simx_opmode_blocking);
[~, handles(9)]=sim.simxGetObjectHandle(clientID,'youBotArmJoint4',sim.simx_opmode_blocking);
[~, handles(10)]=sim.simxGetObjectHandle(clientID,'youBotGripperJoint1',sim.simx_opmode_blocking);
[~, handles(11)]=sim.simxGetObjectHandle(clientID,'youBotGripperJoint2',sim.simx_opmode_blocking);
[~, handles(12)]=sim.simxGetObjectHandle(clientID,'visionSensor',sim.simx_opmode_blocking); userGui = gui;
```

Рисунок 14. Получение идентификаторов компонентов робота в файле `main.m` с разделением идентификаторов пальцев

Также для управления пальцами схвата необходимо добавить графический интерфейс для управления пальцами. Дополненный графический интерфейс представлен на рисунке 15: для осуществления функции управления нужными звеньями добавлен блок Gripper Control.

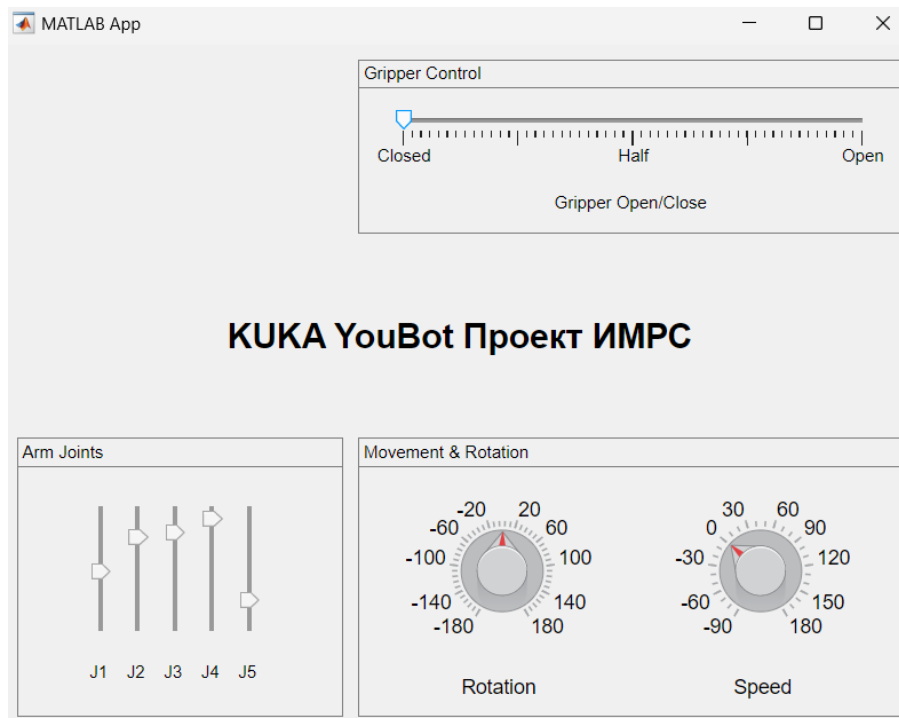


Рисунок 15. Дополненный графического интерфейса для реализации управления схватом

На рисунке 16 представлено объявление компонентов графического интерфейса для управления захватом робота: панель для размещения элементов, разметка для их расположения, метку с названием и слайдер для регулировки степени открытия захвата. Это необходимо для создания интуитивного управления, позволяющего плавно открывать и закрывать gripper робота.

<code>GripperPanel</code>	<code>matlab.ui.container.Panel</code>
<code>GridLayout5</code>	<code>matlab.ui.container.GridLayout</code>
<code>GripperLabel</code>	<code>matlab.ui.control.Label</code>
<code>GripperSlider</code>	<code>matlab.ui.control.Slider</code>

Рисунок 16. Компоненты графического интерфейса, отвечающие за схват

Код на рисунке 17 реализует управление захватом (gripper) робота, преобразуя значение от 0 до 1 с ползунка графического интерфейса в физические позиции двух пальцев схвата в симуляторе. Функция проверяет доступность управляющих идентификаторов захвата, затем вычисляет линейные перемещения для пальца. Палец движется вперед на расстояние до 0.025 метра, что позволяет открывать и закрывать захват. Команды передаются в симулятор CoppeliaSim, обеспечивая плавное и точное

управление схватом для захвата и удержания объектов.

```
function setGripperPosition(app, value)

    if length(app.handles) < 11 || app.handles(10) == -1 || app.handles(11) == -1
        fprintf('ERROR: Gripper handles not found!\n');
        return;
    end

    maxMovement = 0.025;
    position1 = value * maxMovement; % 0 -> 0.025 м

    position2 = -value * maxMovement; % 0 -> -0.025 м

    app.sim.simxSetJointPosition( ...
        app.clientID, ...
        app.handles(10), ...
        position1, ...
        app.sim.simx_opmode_streaming ...
    );

    app.sim.simxSetJointPosition( ...
        app.clientID, ...
        app.handles(11), ...
        position2, ...
        app.sim.simx_opmode_streaming ...
    );
end
```

Рисунок 17. Реализация функции управления схватом

На рисунке 18 представлен код, который создает графический интерфейс для управления захватом робота в MATLAB-приложении. Он добавляет на панель управления слайдер, который позволяет пользователю плавно регулировать степень открытия захвата от полностью закрытого до полностью открытого состояния. Слайдер имеет подписи "Closed", "Half" и "Open", что делает управление захватом удобным. Когда пользователь перемещает этот слайдер, его движение преобразуется в команды, которые отправляются в симулятор CoppeliaSim для физического управления пальцами захвата робота.


```

% Create GripperPanel
app.GripperPanel = uipanel(app.GridLayout);
app.GripperPanel.Title = 'Gripper Control';
app.GripperPanel.Layout.Row = 1;
app.GripperPanel.Layout.Column = 2;

% Create GridLayout5
app.GridLayout5 = uigridlayout(app.GripperPanel);
app.GridLayout5.ColumnWidth = {'1x'};
app.GridLayout5.RowHeight = {'1x', 22};

% Create GripperLabel
app.GripperLabel = uilabel(app.GridLayout5);
app.GripperLabel.HorizontalAlignment = 'center';
app.GripperLabel.Layout.Row = 2;
app.GripperLabel.Layout.Column = 1;
app.GripperLabel.Text = 'Gripper Open/Close';

% Create GripperSlider
app.GripperSlider = uislider(app.GridLayout5);
app.GripperSlider.Limits = [0 1];
app.GripperSlider.MajorTicks = [0 0.25 0.5 0.75 1];
app.GripperSlider.MajorTickLabels = {'Closed', '', 'Half', '', 'Open'};
app.GripperSlider.ValueChangingFcn = createCallbackFcn(app, @GripperSliderValueChanging, true);
app.GripperSlider.Layout.Row = 1;
app.GripperSlider.Layout.Column = 1;
app.GripperSlider.Value = 0.5;

```

Рисунок 18. Реализация графического интерфейса для управления схватом

На рисунке 19 представлено исходное открытое положение схвата робота, демонстрирующее начальную конфигурацию системы, при которой пальцы захвата разведены и готовы к выполнению операции взятия объекта. Рисунок 20 иллюстрирует конечный результат работы программы, где схват успешно закрылся, надежно захватив целевой объект, что подтверждает корректную работу алгоритма управления и способность робота выполнять манипуляционные задачи. Сравнение этих двух состояний наглядно показывает эффективность реализованной системы управления захватом, которая позволяет плавно переходить из открытого в закрытое положение с точным позиционированием пальцев для удержания объектов.

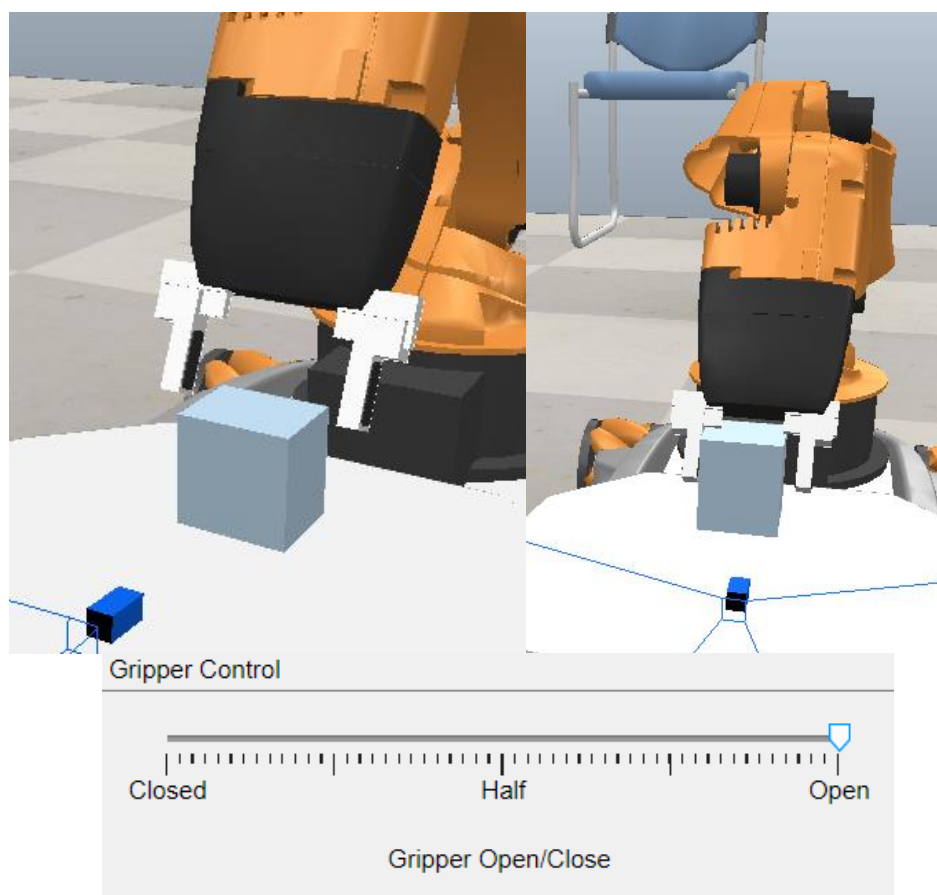


Рисунок 19. Демонстрация исходного положения схвата (открытое положение)

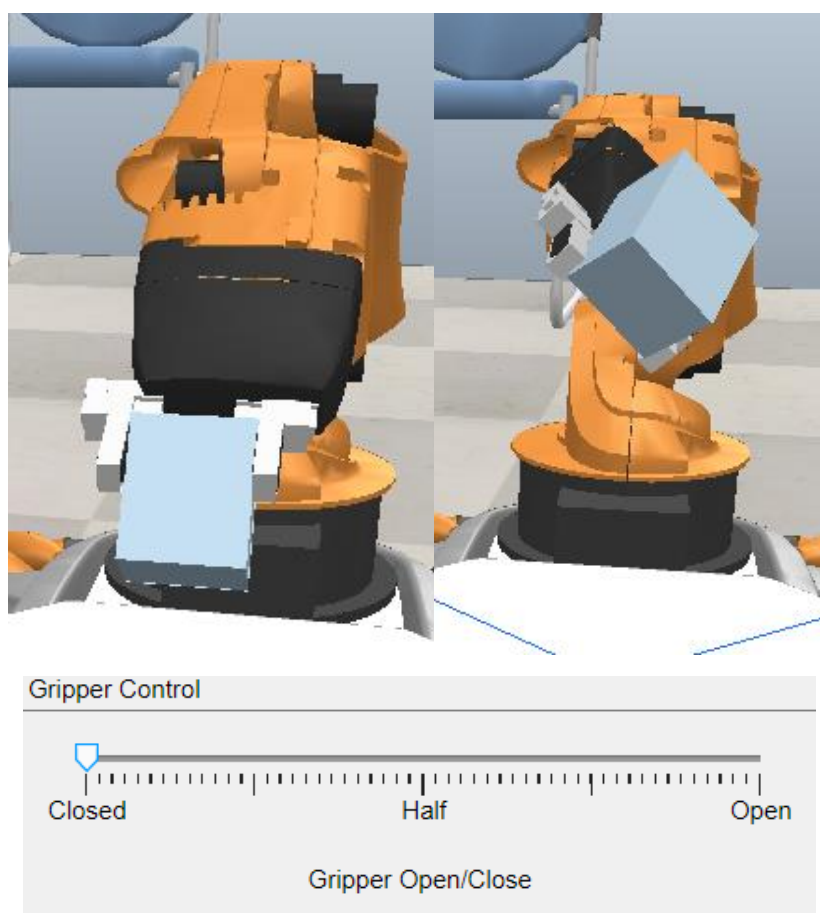


Рисунок 20. Демонстрация управления положением схвата

В ходе проектной работы на этапе 3 была реализована функция управления захватом робота youBot. Для этого в исходный код были внесены необходимые корректировки, включающие разделение идентификаторов пальцев схвата в файле `main.m` и разработку соответствующего графического интерфейса в приложении MATLAB. Была создана интуитивная панель управления с плавным слайдером, позволяющим регулировать степень открытия захвата от полностью закрытого до полностью открытого состояния. Реализованный алгоритм преобразует перемещения слайдера в точные линейные перемещения пальцев в симуляторе CoppeliaSim с амплитудой до 0.025 метра. Практическая демонстрация работы системы подтвердила её эффективность – захват переходит из открытого положения в закрытое, успешно удерживая целевые объекты, что обеспечивает возможность выполнения роботом манипуляционных операций.

Неполноприводность в контексте данного проекта означает, что схват робота KUKA youBot имеет четыре степени свободы (два сустава на каждом из двух пальцев), но при этом оснащён всего двумя приводами. Это создаёт архитектурное несоответствие: количество управляющих воздействий (два мотора) меньше, чем количество изменяемых параметров системы (четыре угла поворота суставов). В традиционных полноприводных системах каждый сустав имел бы отдельный мотор и систему управления, что обеспечило бы полный контроль над конфигурацией захвата, но одновременно привело бы к усложнению конструкции, увеличению стоимости, массы и энергопотребления.

Решение проблемы неполноприводности в данной работе реализовано на механическом, а не алгоритмическом уровне. Вместо того чтобы компенсировать недостаток приводов сложными вычислениями или дополнительными датчиками, используется принцип пассивной адаптации, заложенный в конструкцию схвата. Два активных привода управляют только базовыми (проксимальными) суставами пальцев. Дистальные (концевые) суставы жёстко связаны с проксимальными через механические передачи

(систему рычагов или зубчатых колёс). Эта связь задаёт фиксированное соотношение между углами поворота: движение дистального сустава является функцией движения проксимального.

ЗАКЛЮЧЕНИЕ

В ходе выполнения проекта была успешно реализована и исследована имитационная модель неполноприводного схвата робота KUKA youBot в среде CoppeliaSim. Работа продемонстрировала практическую возможность создания эффективной системы управления для адаптивного захватного устройства, основанного на принципах неполноприводности.

В ходе данной работы была исследована роботизированная мобильная платформа KUKA youBot; изучены основы кинематики и динамики данного робота.

Ключевым достижением проекта стало установление работоспособной связи между вычислительной средой MATLAB и симулятором CoppeliaSim через Remote API, что позволило осуществлять удалённое управление всеми компонентами роботизированной платформы. Была разработана модульная система управления, включающая независимое управление мобильной платформой, манипулятором и двухпальцевым схватом. Особое внимание уделялось реализации управления схватом – была создана система, позволяющая плавно регулировать степень открытия захвата.

Проведённые эксперименты подтвердили теоретические положения о преимуществах неполноприводной архитектуры. Схват продемонстрировал способность к пассивной адаптации при захвате объектов различной геометрии, обеспечивая надёжное удержание без необходимости точного позиционирования каждого сустава. Визуализация процессов захвата в симуляторе наглядно показала, как механические связи между суставами позволяют системе автоматически распределять контактные силы.

Несмотря на достигнутые результаты, работа выявила и определённые ограничения, характерные для неполноприводных систем. В частности, была отмечена сниженная точность позиционирования по сравнению с полноприводными аналогами и зависимость эффективности захвата от параметров трения и геометрии объекта.

СПИСОК ЛИТЕРАТУРЫ

1. A Study of Accuracy and Time Delay for Bilateral Master-Slave Industrial Robotic Arm Manipulator System // Proceedings of the International Conference on Industrial Engineering, Applications and Manufacturing (ICIEAM). – 2022. – С. 1-6. – DOI 10.1109/ICIEAM54945.2022.9787111.
2. Condition Monitoring and Cloud-based Energy Analysis for Autonomous Mobile Manipulation - Smart Factory Concept with LUHbots // Procedia CIRP. – 2021. – Vol. 103. – С. 208-213. – DOI 10.1016/j.procir.2021.10.033.
3. Constraint-Based Model Predictive Control for Holonomic Mobile Manipulators // IEEE Robotics and Automation Letters. – 2018. – Vol. 3, № 4. – С. 4231-4238. – DOI 10.1109/LRA.2018.2864348.
4. Developing of KUKA youBot Software for Education Process // *Proceedings of the 18th International Conference on Mechatronics - Mechatronika (ME)*. – 2018. – С. 1-7.
5. Investigation of Human-Robot Interface Performance in Household Environments // International Journal of Social Robotics. – 2023. – Vol. 15, № 3. – С. 523-539. – DOI 10.1007/s12369-021-00767-9.
6. Redundancy modelling and resolution for robotic mobile manipulators: a general approach // Robotica. – 2016. – Vol. 34, № 11. – С. 2466-2487. – DOI 10.1017/S026357471500009X.
7. Symbiotic relationship between robots - a ROS ARDrone/YouBot library // Proceedings of the 11th International Conference on Informatics in Control, Automation and Robotics (ICINCO). – 2014. – Vol. 2. – С. 462-469.
8. Официальный сайт KUKA youBot: раздел для разработчиков [Электронный ресурс]. – URL: <http://www.youbot->

store.com/developers/kuka-youbot-kinematics-dynamics-and-3d-model-81 (дата обращения: 18.11.2025).

9. Sonderegger, E. Репозиторий программного обеспечения для KUKA youBot (kuka_youbot) [Электронный ресурс] / E. Sonderegger // GitHub.
– URL: https://github.com/esonderegger/kuka_youbot (дата обращения: 18.12.2025).